

Modul Media Programming, Lehrveranstaltung AV Programming



Prof. Dr. Eva Wilk

HAW Hamburg, Fakultät INF, Studiengang Medieninformatik

Thema 6 - Übersicht

AV-Bearbeitung und Synchronisation

- Audio bearbeiten
- Video bearbeiten
- Musik zu einem Clip mischen
- Offset und Drift unterscheiden und reparieren
- Wahrnehmung von A/V-Versatz mit eigenem Demo-Material prüfen

Thema 5 - Lernziel

Die Studierenden bearbeiten Audio- und Videodaten mit Python, OpenCV und FFmpeg. Sie wählen geeignete Bearbeitungsschritte für typische Aufgaben der AV-Bearbeitung aus und beurteilen deren Auswirkungen auf Qualität, Zeitbezug und Synchronisation.

Asynchronität

- Audio vor Video oder
 - Video vor Audio
- mit
- konstantem Offset oder
 - Drift über die Zeit

Asynchronität

Inhalt von J.248: betriebliche Überwachung von Video-zu-Audio-Verzögerung in Fernsehverteilungsketten

Ausgangspunkt: Lip-Sync-Fehler entstehen vor allem dann, wenn Audio und Video in der Pipeline getrennt verarbeitet werden

Kritisch: videoseitige Verzögerungen, etwa durch Frame-Synchronizer, datenreduzierende Encoder/Decoder und komplexe Bildverarbeitung

Ziel der Empfehlung: Anforderungen an ein Monitoring, mit dem sich solche Fehler im laufenden Betrieb erkennen und minimieren lassen

Asynchronität

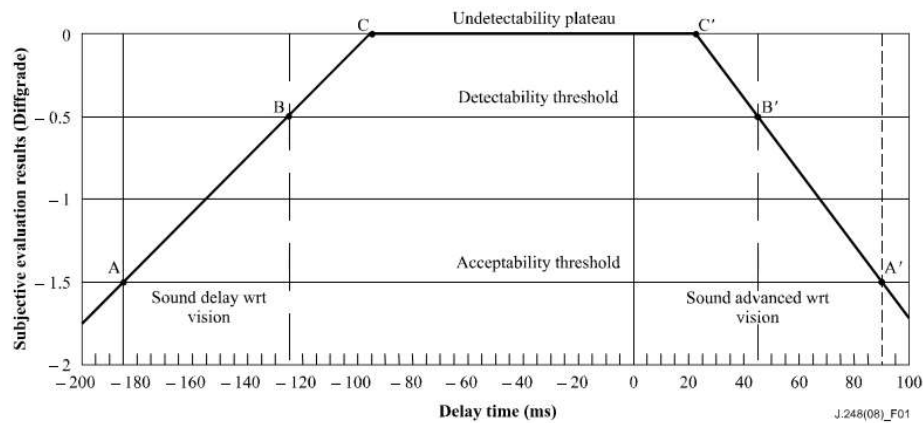
Formale Empfehlungen zur Beobachtung von Video-zu-Audio-Verzögerung:
ITU-T J.248 (06/2008) „*Requirements for operational monitoring of video-to-audio delay in the distribution of television programs*“

https://www.itu.int/rec/dologin_pub.asp?lang=e&id=T-REC-J.248-200806-1!!!PDF-E&type=items

- Lip-Sync: muss quantitativ beurteilbar sein.
- Monitoring: möglichst im laufenden Betrieb, schnell und möglichst nahe an der Fehlerquelle
- Wichtig: Unterscheidung zwischen konstantem Offset und zeitlichem Drift
- Grobe Orientierung:
 - Wahrnehmungsgrenzen: **+45 ms bei Audio vor Video** und **-125 ms bei Audio nach Video**
 - Akzeptanz-Grenzen: **90 ms .. -185 ms**
 - Engere Grenzen für impulshafte Ereignisse

Asynchronität

Each threshold is set to the lead or lag value at which 50% of the assessors cast the higher integer score and 50% cast the lower integer score in the 5 grades assessment scale.



https://www.itu.int/rec/dologin_pub.asp?lang=e&id=T-REC-J.248-200806-1!!PDF-E&type=items

Aufgabe 12

Untersuchen Sie unterschiedliche Versatz-Werte zwischen A und V auf ihre Wahrnehmbarkeit. Es stehen zwei Videos dafür zur Auswahl.

Vorgehen:

1. gewünschten Offset einlesen und in ms umrechnen
2. Dauer des Videos feststellen
3. je nach Vorzeichen des Offsets Audiospur nach vorn oder nach hinten ziehen
4. das bearbeitete File unter neuem Namen speichern

Dauer: 15 Minuten

Aufgabe 12

```
video_duration = get_duration(input_file)
```

Positiver Offset: Audio wird verzögert.

Video: |-----|
Audio: |-----|
 Stille

```
„-af“, Audiofilter → auf die Audiospur soll ein Filter angewendet werden  
f"adelay={delay_ms}:all=1" alle Audiokanäle werden um delay_ms verschoben  
f"atrim=duration={video_duration}, " Ende abschneiden  
f"asetpts=PTS-STARTPTS" Zeitstempel der Audiospur neu setzen
```

Aufgabe 12

Negativer Offset: Anfang von Audio wird abgeschnitten

Video: |-----|
Audio: |-----|
 Anfang fehlt

```
„-af, “  
f"atrim=start={audio_start}, " entfernt die ersten |audio_start| ms der Audiospur  
f"asetpts=PTS-STARTPTS, " Zeitstempel der Audiospur wird auf null gesetzt  
f"apad, " hinten Stille auffüllen  
f"atrim=duration={video_duration}" Audiospur auf ursprüngliche Videolänge begrenzen  
"-t", str(video_duration), begrenzt die Gesamtdauer der Ausgabedatei
```

Aufgabe 12

Ottos Mops

<https://www.youtube.com/shorts/CMSzFGCNISM>



Wilhelm Busch

<https://www.youtube.com/shorts/zhNmH9ldews>

Youtube-Download: https://github.com/yt-dlp/yt-dlp?utm_source=chatgpt.com#installation

Audio-Bearbeitung: Pegel, Pausen, Filter

- Pegel anheben `volume`
- sehr leise Stellen entfernen `silenceremove`
- tieffrequente Störungen abschwächen `highpass`
- hohe Störgeräusche abschwächen `lowpass`

Beispiel 19: Audio bearbeiten

- WAV lesen
- +6 dB Gain
- 20 ms-Fenster bilden
- RMS berechnen
- Abschnitte mit Signalen unter einem gegebenen Schwellwert herausschneiden

Beispiel 19: Audio bearbeiten

```
...
sr, x = read_wav_mono("speech.wav")

gain_db = 6
gain = 10 ** (gain_db / 20)
x_boost = np.clip(x.astype(np.float32) * gain, -32768, 32767).astype(np.int16)

frame_len = int(0.02 * sr)
usable = len(x_boost) // frame_len * frame_len
frames = x_boost[:usable].reshape(-1, frame_len)

rms = np.sqrt(np.mean(frames.astype(np.float32) ** 2, axis=1))
threshold = 500
keep = rms > threshold

x_short = frames[keep].reshape(-1)
write_wav_mono("speech_edit.wav", sr, x_short)
```

Video-Bearbeitung: Schneiden, Bildwirkung verändern

- Zeitfenster auswählen
- Helligkeit ändern
- Graustufen oder Farbwirkung ändern

Befehle: `CAP_PROP_FPS`, `CAP_PROP_FRAME_COUNT`, `CAP_PROP_POS_MSEC`

Technische Größen

- FPS
- Framezahl
- Zeit = $\text{Frameindex} / \text{FPS}$
- Export mit `VideoWriter`

Beispiel 20: Video bearbeiten

- Zeitfenster Sekunde 2 bis Sekunde 6
- Umwandlung in Graustufen
- Helligkeit um den Wert 30 erhöhen, dabei Clipping vermeiden
- Export mit `VideoWriter`

Beispiel 20: Video bearbeiten

```
...
start_s = 2.0
end_s = 6.0
frame_idx = 0

while True: #Schleife bis Dateieinde
    ok, frame = cap.read()
    if not ok:
        break

    t = frame_idx / fps

    if start_s <= t < end_s:
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) #Graustufenbild mit nur 1 Kanal
        gray3 = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR) # Umwandeln in 3kanaliges Bild, gleicher Grauwert auf
                                                    jedem Kanal
        bright = np.clip(gray3.astype(np.int16) + 30, 0, 255).astype(np.uint8)
        writer.write(bright)
    frame_idx += 1
```

out[:, :, 0]: Blaukanal aller Pixel, wird begrenzt auf Werte zwischen und und 255

Aufgabe 13:

Schneiden Sie aus „Ottos Mops.mp4“ den Vorspann heraus.

Hinweis: verwenden Sie ffmpeg in einem Python-Subprocess.

```
subprocess.run([
    "ffmpeg", "-y",
    ...
    output_file
], check=True)
```

Fmpeg-Dokumentation: <https://ffmpeg.org/ffmpeg.html>.

Nützliche Optionen: `-ss`, `-t`, `-c` (Abschnitt 5.4, Main Options)

Dauer: 15 Minuten

AV-Clip mit Musik ergänzen

Entscheiden:

- Dauer der Musik?
- Pegel der Musik? (dominant oder nur Hintergrund?)
- Video neu codieren oder kopieren?

FFmpeg-Filter:

- volume
- weights, normalize
- duration
- amix *Mischen mehrerer Audioeingänge*
- -c:v copy *Bildstrom bleibt unverändert*

Beispiel 21: Musik unter Videoclip

```
...
clip = "Ottos_Mops.mp4"
musik = "music.mp3"
ausgabe = "clip_mit_musik.mp4"

lautstaerke_original = 1.0
lautstaerke_musik = 0.20
dauerregel = "first" # alternativ: longest, shortest

filter_komplex = (
    f"[0:a]volume={lautstaerke_original}[orig];"
    f"[1:a]volume={lautstaerke_musik}[musik];"
    f"[orig][musik]amix=inputs=2:duration={dauerregel}:normalize=0[mix]"
)

cmd = [
    "ffmpeg", "-y", "-i", clip, "-i", musik, "-filter_complex", filter_komplex,
    "-map", "0:v", "-map", "[mix]", "-c:v", "copy", "-c:a", "aac", ausgabe
]
```

Offset oder Drift reparieren

Typische Synchronisations-Fehler:

- Offset – Audio oder Video starte zu früh
- Drift – Start passt, Ende nicht

Werkzeuge:

- `setpts` / `asetpts` *ändern der Presentation Timestamps*
- `atempo` *Werkzeug für lineare Drift*
- `aresample=async=...` *kann Audio an Timestamps anpassen, bei Bedarf strecken, stauchen, Stille einfügen, Audio entfernen*

Offset oder Drift reparieren

```
# konstanter Offset: Video 80 ms später  
ffmpeg -i in.mp4 -filter_complex "[0:v]setpts=PTS+0.08/TB[v]" -map "[v]" -map 0:a out.mp4
```

Offset oder Drift reparieren

```
# konstanter Offset: Video 80 ms später  
ffmpeg -i in.mp4 -filter_complex "[0:v]
```

Eingabedatei

Videostream
aus Eingabe 0
(also in. mp4)

```
setpts=PTS+0.08/TB[v]"
```

Addiere 0.08 Sekunden zu
jedem Video-Timestamp

```
-map "[v]" -map 0:a
```

in die Ausgabe: gefilterter
Videostream v und Audio aus
Input 0

```
out.mp4
```

Name Ausgabedatei

Der Befehl verschiebt nicht das Bildmaterial, sondern den Zeitstempel.

Dadurch verschiebt er den Videostream um 80 ms nach hinten.

Offset oder Drift reparieren

```
# kleiner Drift: Audio leicht verlangsamen  
ffmpeg -i in.mp4 -filter:a "atempo=0.9942" out.mp4
```

Beispiel-Rechnung:

- Dauer Video: 60.00 s
- Dauer Audio: 59.65 s
- Differenz: 0.35 s
- Faktor für Audio: $59.65 / 60.00 = 0.9942$

Einfache Videoeffekte mit Python/OpenCV

1. Weichzeichnen (Blur)

Ein Videoausschnitt wird mit `GaussianBlur` weichgezeichnet.

2. Überblendung zwischen zwei Clips

Die letzten Frames von Clip 1 werden mit den ersten Frames von Clip 2 gemischt.

Einfache Videoeffekte mit Python/OpenCV

1. Weichzeichnen (Blur)

Ein Videoausschnitt wird mit `GaussianBlur` weichgezeichnet.

```
out = cv2.GaussianBlur(frame, (15, 15), 0)
```

frame: Eingabebild

(15, 15): Größe der berücksichtigten Nachbarschaft

0: sigmaX. wenn 0: OpenCV bestimmt die Breite der Gauß-Verteilung automatisch

Ergebnis: Das Bild wird weichgezeichnet, Kanten und feine Details werden abgeschwächt

Verändern Sie die Stärke des Blur-Effekts und beschreiben Sie die Bildwirkung.
Werte z.B.: (5, 5), (15, 15) und (31, 31)

Die Kernelgröße muss bei GaussianBlur ungerade sein, damit der Filter ein eindeutiges Zentrum hat (wegen: 1 Pixel in der Mitte).

Einfache Videoeffekte mit Python/OpenCV

2. Überblendung zwischen zwei Clips

Die letzten Frames von Clip 1 werden mit den ersten Frames von Clip 2 gemischt.

`cv2.addWeighted(frame1, alpha, frame2, beta, 0)` mischt zwei Bilder.

Am Anfang der Überblendung dominiert `clip1`, am Ende `clip2`.

Wichtig: Beide Videos sollten möglichst die gleiche Auflösung und die gleiche Framerate haben.

Einfache Audioeffekte mit Python und wave

1. Audio-Crossfade

Zwei Audiodateien werden weich ineinander überblendet: Der erste Clip wird am Ende ausgeblendet, der zweite gleichzeitig eingeblendet.

2. Gate

Leise Abschnitte werden abgesenkt oder stummgeschaltet.
Die Zeitstruktur bleibt dabei erhalten.

Einfache Audioeffekte mit Python und wave

1. Audio-Crossfade

Zwei Audiodateien werden weich ineinander überblendet: Der erste Clip wird am Ende ausgeblendet, der zweite gleichzeitig eingeblendet.

Ansatz: a = erstes Audiosignal, b = zweites Audiosignal →
Anfang von a unverändert,
dann Ende von a und Anfang von b überblenden,
danach Rest von b

Wichtig: die Abtastfrequenzen von a und b müssen gleich sein.

Wann ist ein harter Schnitt sinnvoll – und wann wirkt eine weiche Pegeländerung fachlich und gestalterisch überzeugender?

Einfache Audioeffekte mit Python und wave

1. Audio-Crossfade

```
...
fade_dur_s = 2.0
fade_n = int(fade_dur_s * sr)

fade_out = np.linspace(1.0, 0.0, fade_n, endpoint=True)
fade_in = np.linspace(0.0, 1.0, fade_n, endpoint=True)

head = a[:-fade_n] # a bis Beginn von fade
a_tail = a[-fade_n:] * fade_out # Rest von a wird ausgeblendet
b_head = b[:fade_n] * fade_in # Anfang von b wird eingefadet
cross = a_tail + b_head # die Fade-Abschnitte werden übereinandergelegt
tail = b[fade_n:] # Rest von b nach dem Einfaden
```

Einfache Audioeffekte mit Python und wave

2. Gate

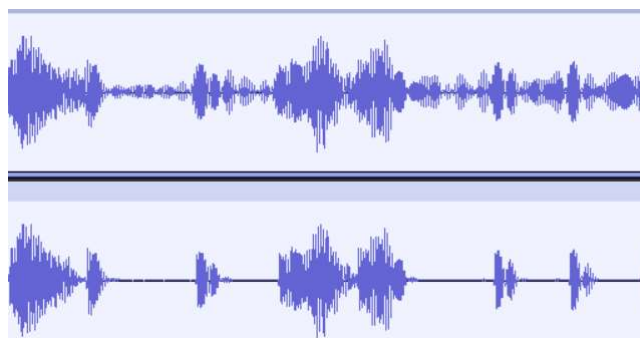
Leise Abschnitte werden abgesenkt oder stummgeschaltet. Die Zeitstruktur bleibt dabei erhalten.

- Das Signal wird in kurze Zeitfenster zerlegt
- Für jedes Fenster wird ein Effektivwert (RMS) berechnet
- Je nach RMS-Wert bekommt das Fenster die Verstärkung
 - 1.0 → bleibt unverändert
 - oder 0.0 → wird stumm
- Alle Fenster werden wieder zum Signal zusammengesetzt

Einfache Audioeffekte mit Python und wave

2. Gate

Leise Abschnitte werden abgesenkt oder stummgeschaltet. Die Zeitstruktur bleibt dabei erhalten.



Einfache Audioeffekte mit Python und wave

2. Gate

```
...
parts = []

i = 0
while i < len(x):
    frame = x[i:i + frame_len]
    rms = np.sqrt(np.mean(frame ** 2))

    if rms > threshold:
        parts.append(frame)
    else:
        parts.append(frame * gate_gain)

    i += frame_len

y = np.concatenate(parts)
```

Einordnung

Bearbeitungseffekte verändern das audiovisuelle Material gezielt im Hinblick auf Wahrnehmung, Gestaltung oder technische Verwendbarkeit.

Beispiele in diesem Termin sind:

- Audio: Pegelanhebung, Hochpass, Entfernen leiser Abschnitte, Musikmischung
- Video: Ausschnitt, Helligkeitsänderung, Graustufen, Blur, Überblendung

Analyseoperationen: zur Beschreibung, Messung oder Bewertung von Material.

Beispiele in diesem Termin sind:

- Messung von Audio-/Videodauer
- Beurteilung von Offset und Drift
- Einschätzung der Wahrnehmbarkeit von A/V-Versatz

Fehlerquellen bei AV-Bearbeitung und Synchronisation

1. Audio

Pegelanhebung führt zu Clipping

Schwellwerte für das Entfernen leiser Abschnitte sind zu aggressiv

Filtergrenzfrequenzen sind unpassend gewählt

Musik überdeckt Sprache statt sie nur zu ergänzen

2. Video

falscher Zeitbereich wird exportiert

Framerate wird nicht korrekt berücksichtigt

Farbraum oder Kanalreihenfolge werden missverstanden

Helligkeitsänderungen führen zu sichtbarem Übersteuern

Fehlerquellen bei AV-Bearbeitung und Synchronisation

...

3. AV-Synchronisation

Offset und Drift werden verwechselt

ein Problem wird subjektiv gehört, aber nicht technisch überprüft

Video und Audio werden getrennt verändert, ohne ihre gemeinsame Zeitbasis mitzudenken

Reparaturmaßnahmen korrigieren nur den Anfang, nicht den Verlauf